

Herança, Classes Abstratas e Polimorfismo

Reaproveitando código e generalizando comportamentos em Java

Linguagem de Programação II

Objetivos da aula

- Entender por que duplicar código entre classes parecidas é um problema.
- Usar **herança** (`extends`) para reaproveitar atributos e métodos.
- Entender a diferença entre *herdar* um atributo e *acessar* um atributo privado diretamente.
- Sobrescrever métodos com `@Override` e reaproveitar comportamento com `super`.
- Entender e aplicar **polimorfismo**: tratar objetos diferentes de forma uniforme.
- Criar e usar **classes e métodos abstratos**.
- Conhecer as **definições formais** de herança, reuso, sobrescrita, polimorfismo e classe abstrata.
- Praticar com hierarquias de `Animal`, `Forma`, `Veiculo` e `Funcionario`.

Parte 1 – Introdução e Motivação

Por que precisamos de herança?

Problema: código duplicado

```
public class Cachorro {  
    private String nome;  
    private int idade;  
    private String raca;  
    // construtor, getters, toString...  
}  
  
public class Gato {  
    private String nome;  
    private int idade;  
    private boolean castrado;  
    // construtor, getters, toString...  
}
```

nome e idade aparecem duas vezes. Se quisermos adicionar um novo atributo comum (ex.: peso), precisamos alterar as duas classes.

Reuso de código

É a prática de escrever uma funcionalidade **uma única vez** e utilizá-la em múltiplos contextos, evitando duplicação e facilitando a manutenção.

- Já vimos uma forma de reuso: a **composição** (uma classe *tem um* atributo de outra classe, como `Cliente` e `Endereco`).
- Hoje vamos conhecer outra forma de reuso: a **herança**.

A solução: herança

- Criamos uma **superclasse** (classe-mãe) com tudo que é comum.
- As **subclasses** (classes-filhas) herdam esses atributos e métodos, e adicionam o que for específico.
- Relação "é um": um Cachorro é um Animal; um Gato é um Animal.
- Em Java, usamos a palavra-chave `extends`.

Herança

É um mecanismo da orientação a objetos em que uma classe (**subclasse**) adquire atributos e métodos de outra classe (**superclasse**), podendo reaproveitá-los e estendê-los com novos comportamentos.

- Palavra-chave em Java: `extends`.
- Representa sempre uma relação "**é um**" (*is-a*).

Definição: Superclasse e Subclasse

Superclasse (classe-mãe)

Classe mais **geral**, que concentra os atributos e métodos comuns a várias classes relacionadas.

Subclasse (classe-filha)

Classe mais **específica**, que herda da superclasse e adiciona (ou sobrescreve) o que for particular a ela.

Sintaxe básica da herança

```
public class Subclasse extends Superclasse {  
    // atributos e metodos extras  
}
```

- Em Java, uma classe só pode ter **uma** superclasse direta (não existe herança múltipla de classes).
- A subclasse herda automaticamente os métodos public (e protected) da superclasse.

Parte 2 – Herança na Prática

`extends`, `super()` e atributos privados

Exemplo: Superclasse Animal

```
public class Animal {  
    private String nome;  
    private int idade;  
  
    public Animal(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    public String getNome() { return nome; }  
    public int getIdade() { return idade; }  
  
    @Override  
    public String toString() {  
        return "Animal:␣" + nome + "␣|␣Idade:␣" +  
            idade;  
    }  
}
```

Atenção

- Mesmo um Cachorro sendo um `Animal`, ele **não pode** acessar `nome` ou `idade` diretamente – eles são `private` em `Animal`.
- Para inicializá-los, a subclasse usa `super(...)` dentro do seu construtor.
- Para lê-los, usamos os *getters* herdados: `getNome()`, `getIdade()`.
- Existe o modificador `protected`, que permite acesso direto das subclasses – mas vamos manter `private` + *getters*, seguindo o mesmo padrão de encapsulamento das aulas anteriores.

Exemplo: Subclasse Cachorro extends Animal

```
public class Cachorro extends Animal {  
    private String raca;  
  
    public Cachorro(String nome, int idade, String  
        raca) {  
        super(nome, idade);  
        this.raca = raca;  
    }  
  
    public String getRaca() { return raca; }  
  
    @Override  
    public String toString() {  
        return super.toString() + "└┘Raca:└┘" + raca;  
    }  
}
```

Exemplo: usando Cachorro no Main

```
public class Main {  
    public static void main(String[] args) {  
        Cachorro rex = new Cachorro("Rex", 3, "  
            Labrador");  
  
        System.out.println(rex.getNome());  
        System.out.println(rex.getRaca());  
        System.out.println(rex);  
    }  
}
```

Saída:

```
Rex  
Labrador  
Animal: Rex | Idade: 3 | Raca: Labrador
```

O que a subclasse herda automaticamente

- Os atributos privados existem dentro do objeto, mas não podem ser acessados diretamente pelo nome.
- Métodos `public` (e `protected`) são herdados e podem ser chamados normalmente.
- O **construtor não é herdado**: a subclasse precisa do seu próprio construtor, chamando `super(...)`.
- Se `super(...)` não for chamado explicitamente, o Java tenta usar um construtor sem parâmetros da superclasse – e dá erro se ele não existir.

Herança vs. Composição

- **Herança** representa uma relação "**é um**": Cachorro *é um* Animal.
- **Composição** representa uma relação "**tem um**": Cliente *tem um* Endereco.
- Nem tudo deve virar herança – às vezes a composição é a escolha mais natural.
- Regra prática: antes de usar extends, pergunte-se: "*é um*" ou "*tem um*"?

Vantagens:

- **Reuso de código:** escrevemos uma vez na superclasse, usamos em várias subclasses.
- **Organização:** a hierarquia reflete o domínio do problema.
- **Manutenção:** corrigir algo na superclasse corrige em todas as subclasses.

Cuidados:

- Hierarquias muito profundas ficam difíceis de entender.
- Herança cria um acoplamento forte entre superclasse e subclasse.
- Use herança apenas quando existir, de fato, uma relação "é um".

Em poucas palavras

- `extends` cria uma relação "**é um**" entre subclasse e superclasse.
- Atributos privados continuam privados; usamos `super(...)` e *getters*.
- O construtor não é herdado – cada subclasse cria o seu, chamando `super(...)`.
- Herança é uma forma de **reuso de código**, assim como a composição.

Sobrescrita de métodos: `emitirSom()`

- Queremos que cada tipo de animal emita um som diferente.
- Podemos declarar um método em `Animal` e sobrescrevê-lo (`@Override`) em cada subclasse.
- Isso é diferente de `toString()`: aqui, cada subclasse terá um comportamento totalmente próprio.

Definição: Sobrescrita de Método (*Override*)

Sobrescrita (*Override*)

Ocorre quando uma subclasse fornece uma **nova implementação** para um método já definido na superclasse, mantendo a mesma assinatura (nome, parâmetros e retorno).

- A anotação `@Override` não é obrigatória, mas pede ao compilador para checar se realmente existe um método correspondente na superclasse.
- Evita erros bobos, como escrever o nome do método errado.

Exemplo: Animal com emitirSom() genérico

```
public class Animal {
    private String nome;
    private int idade;

    public Animal(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    public String getNome() { return nome; }
    public int getIdade() { return idade; }

    public String emitirSom() {
        return "Som genérico de animal";
    }

    @Override
    public String toString() {
        return "Animal: " + nome + " | Idade: " + idade;
    }
}
```

Exemplo: Cachorro sobrescrevendo emitirSom()

```
public class Cachorro extends Animal {
    private String raca;

    public Cachorro(String nome, int idade, String raca) {
        super(nome, idade);
        this.raca = raca;
    }

    public String getRaca() { return raca; }

    @Override
    public String emitirSom() {
        return "Au_au!";
    }

    @Override
    public String toString() {
        return super.toString() + " | Raca: " + raca;
    }
}
```

Exemplo: Gato extends Animal

```
public class Gato extends Animal {
    private boolean castrado;

    public Gato(String nome, int idade, boolean castrado) {
        super(nome, idade);
        this.castrado = castrado;
    }

    public boolean isCastrado() { return castrado; }

    @Override
    public String emitirSom() {
        return "Miau!";
    }

    @Override
    public String toString() {
        return super.toString() + " | Castrado: " + castrado;
    }
}
```

Exemplo: Passaro extends Animal

```
public class Passaro extends Animal {
    private boolean podeVoar;

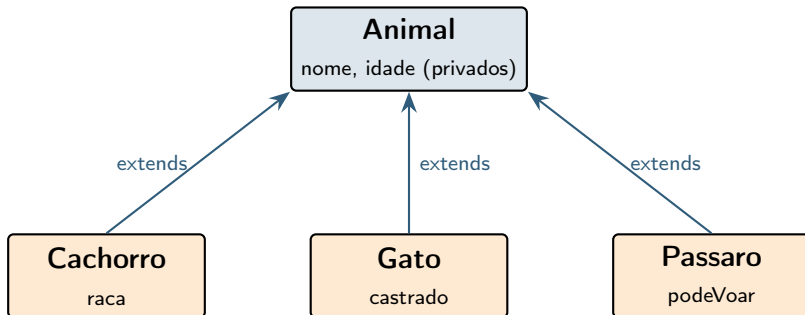
    public Passaro(String nome, int idade, boolean podeVoar) {
        super(nome, idade);
        this.podeVoar = podeVoar;
    }

    public boolean isPodeVoar() { return podeVoar; }

    @Override
    public String emitirSom() {
        return "Piu_piu!";
    }

    @Override
    public String toString() {
        return super.toString() + " | Pode voar: " + podeVoar;
    }
}
```


Hierarquia visual: Animal



Cada subclasse herda `nome` e `idade`, e sobrescreve `emitirSom()` à sua maneira.

Parte 3 – Polimorfismo

Um objeto, muitos comportamentos

Polimorfismo: tratando objetos diferentes de forma uniforme

- **Poli** = muitas, **morfo** = formas → "muitas formas".
- Podemos usar uma referência do tipo `Animal` para guardar um Cachorro, um Gato ou um Passaro.
- Ao chamar um método sobrescrito (como `emitirSom()`), o Java executa a versão da **classe real** do objeto, não da referência.
- Essa decisão acontece em tempo de execução – chamamos isso de *ligação tardia*.

Definição: Polimorfismo

Polimorfismo

É a capacidade de um mesmo método se comportar de maneiras diferentes, dependendo do objeto que o executa, permitindo tratar objetos de subclasses distintas por meio de uma **referência comum** da superclasse.

- Em Java, o polimorfismo de sobrescrita é resolvido **em tempo de execução**.
- É um dos pilares da orientação a objetos, junto com encapsulamento e herança.

Exemplo: vetor de Animal (polimorfismo)

```
public class Main {  
    public static void main(String[] args) {  
        Animal[] animais = new Animal[3];  
  
        animais[0] = new Cachorro("Rex", 3, "Labrador");  
        animais[1] = new Gato("Mimi", 2, true);  
        animais[2] = new Passaro("Piu", 1, true);  
  
        for (int i = 0; i < animais.length; i++) {  
            System.out.println(animais[i].getNome() +  
                "␣diz:␣" + animais[i].emitirSom());  
        }  
    }  
}
```

Exemplo: saída do vetor polimórfico

```
Rex diz: Au au!  
Mimi diz: Miau!  
Piu diz: Piu piu!
```

- O vetor é declarado como `Animal[]`, mas cada posição guarda um objeto de uma subclasse diferente.
- `animais[i].emitirSom()` chama a versão correta automaticamente, conforme o tipo real do objeto.

Pense um pouco

Se a classe Cachorro **não** sobrescrevesse emitirSom(), qual versão do método seria executada ao chamar `rex.emitirSom()`?

- Dica: pense em como o Java procura o método – primeiro na classe do objeto, depois nas superclasses.

Por que isso é útil?

- Podemos escrever um laço genérico que funciona para qualquer `Animal`, atual ou futuro.
- Se criarmos uma nova subclasse (ex.: `Peixe`), o mesmo laço continua funcionando, sem precisar mudar o `Main`.
- Isso deixa o código mais flexível, organizado e fácil de estender.

Em poucas palavras

- Uma referência de superclasse pode apontar para qualquer subclasse.
- O método executado é sempre o da **classe real** do objeto.
- Isso permite escrever código genérico (laços, vetores) que funciona com várias subclasses.

Parte 4 – Classes Abstratas

Obrigando as subclasses a fazerem a parte delas

Problema: Animal genérico não faz muito sentido

- O que `emitirSom()` deveria retornar para um `Animal` "genérico", sem espécie definida?
- Também não faz muito sentido criar um objeto `Animal` puro, sem ele ser de uma espécie específica.
- Queremos impedir isso e **obrigar** cada subclasse a definir seu próprio `som`.

A solução: classes abstratas

- Uma classe **abstrata** representa um conceito geral que não deve ser instanciado sozinho.
- Pode ter atributos, construtor e métodos normais (concretos), como qualquer classe.
- Pode ter **métodos abstratos**: sem corpo, que as subclasses são **obrigadas** a implementar.

Definição: Classe Abstrata

Classe abstrata

É uma classe que **não pode ser instanciada diretamente**, usada para representar um conceito geral e compartilhar código entre suas subclasses. Pode conter métodos concretos e métodos abstratos.

- Palavra-chave em Java: `abstract class`.
- Serve como um "molde" comum para as subclasses concretas.

Definição: Método Abstrato

Método abstrato

É um método declarado **sem implementação** (sem corpo), que obriga toda subclasse concreta a fornecer sua própria implementação.

- Só pode existir dentro de uma classe abstract.
- Termina com ;, em vez de { }.

Sintaxe: abstract class e método abstrato

```
public abstract class Superclasse {  
    public abstract TipoRetorno metodoAbstrato();  
}
```

- A classe é marcada com `abstract` antes de `class`.
- O método abstrato termina com `;` em vez de `{ }` – ele não tem corpo.
- Uma classe com pelo menos um método abstrato **precisa** ser abstrata.

Exemplo: Animal abstrato

```
public abstract class Animal {  
    private String nome;  
    private int idade;  
  
    public Animal(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    public String getNome() { return nome; }  
    public int getIdade() { return idade; }  
  
    public abstract String emitirSom();  
  
    @Override  
    public String toString() {  
        return "Animal:␣" + nome + "␣|␣Idade:␣" + idade;  
    }  
}
```


Atenção: não dá para instanciar uma classe abstrata

```
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Animal("Rex", 3);  
        // ERRO DE COMPILACAO:  
        // Cannot instantiate the type Animal  
    }  
}
```

Atenção

- O compilador impede isso porque a classe está incompleta (tem um método sem implementação).
- Só podemos instanciar as **subclasses concretas** (Cachorro, Gato, Passaro), que implementam emitirSom().

As subclasses continuam iguais – agora com uma obrigação

- As classes Cachorro, Gato e Passaro não mudam: continuam com `extends Animal` e `@Override emitirSom()`.
- A diferença é que, agora, sobrescrever `emitirSom()` **deixa de ser opcional**.
- Se você apagar o `emitirSom()` de Cachorro, o código simplesmente não compila mais.
- O compilador garante que toda subclasse concreta saiba "emitir seu som".

Exemplo: o mesmo Main, agora com Animal abstrato

```
public class Main {  
    public static void main(String[] args) {  
        Animal[] animais = new Animal[3];  
  
        animais[0] = new Cachorro("Rex", 3, "Labrador");  
        animais[1] = new Gato("Mimi", 2, true);  
        animais[2] = new Passaro("Piu", 1, true);  
  
        for (int i = 0; i < animais.length; i++) {  
            System.out.println(animais[i].getNome() +  
                " diz: " + animais[i].emitirSom());  
        }  
    }  
}
```

A saída é exatamente a mesma de antes – o polimorfismo continua funcionando, agora com a garantia da classe abstrata.

Pense um pouco

Por que não bastaria deixar `emitirSom()` em `Animal` com um corpo vazio (`return ;`), em vez de torná-lo abstrato?

- Dica: pense no que acontece se uma subclasse nova **esquecer** de sobrescrever o método.

Quando usar instanceof?

- Às vezes uma subclasse tem um método **próprio**, que não existe na superclasse.
- Para chamá-lo a partir de uma referência da superclasse, precisamos primeiro verificar o tipo real do objeto.
- O operador `instanceof` verifica esse tipo antes de fazer um *cast* (conversão) com segurança.
- Use com moderação: depender muito de `instanceof` costuma indicar que falta um método abstrato bem pensado na superclasse.

Exemplo: instanceof e *downcasting*

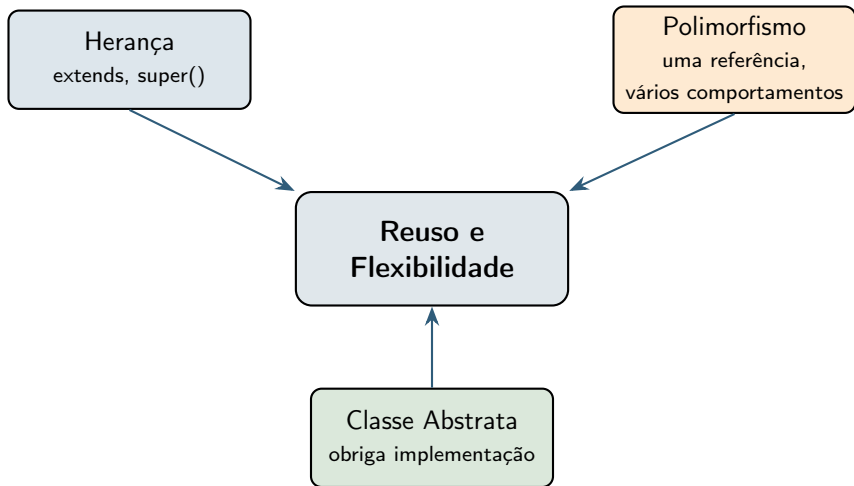
```
for (Animal animal : animais) {  
    if (animal instanceof Passaro) {  
        Passaro p = (Passaro) animal;  
        System.out.println(p.getNome() +  
            " pode voar: " + p.isPodeVoar());  
    }  
}
```

- isPodeVoar() só existe em Passaro, não em Animal.
- O if garante que o *cast* (Passaro) animal só acontece quando é seguro.

Em poucas palavras

- `abstract class`: não pode ser instanciada, mas pode ter atributos, construtor e métodos concretos.
- `abstract method`: sem corpo, obriga as subclasses concretas a implementá-lo.
- Combina muito bem com polimorfismo: garante que todo objeto da hierarquia "sabe se comportar".

Resumo Visual: os três conceitos juntos



Os três conceitos trabalham juntos: a herança organiza, o polimorfismo flexibiliza, e a classe abstrata garante que cada subclasse cumpra seu papel.

Resumo: Herança, Polimorfismo e Classes Abstratas

- **Herança** (`extends`) evita duplicação: atributos e métodos comuns ficam na superclasse.
- Atributos privados da superclasse não são acessados diretamente; usamos `super(...)` e *getters*.
- `@Override` permite que cada subclasse personalize um comportamento; `super.metodo()` reaproveita o da superclasse.
- **Polimorfismo**: uma referência da superclasse pode apontar para qualquer subclasse, e o método correto é escolhido em tempo de execução.
- **Classes abstratas** impedem instâncias "genéricas" e obrigam as subclasses a implementar métodos essenciais.
- Juntos, esses três conceitos tornam o código mais organizado, reaproveitável e fácil de estender.

Parte 5 – Prática Guiada

Colocando a mão na massa

Problema:

- Crie uma classe abstrata `Forma` com:
 - atributo privado `nome`;
 - construtor e *getter*;
 - método abstrato `calcularArea()`;
 - `toString()` concreto, usando `calcularArea()`.
- Crie as subclasses `Circulo` (com raio) e `Retangulo` (com largura e altura).
- No `Main`, crie um vetor de `Forma`, imprima cada uma e some todas as áreas.

Gabarito: Classe abstrata Forma

```
public abstract class Forma {  
    private String nome;  
  
    public Forma(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() { return nome; }  
  
    public abstract double calcularArea();  
  
    @Override  
    public String toString() {  
        return nome + " | Area: " + calcularArea();  
    }  
}
```

Gabarito: Classe Circulo

```
public class Circulo extends Forma {  
    private double raio;  
  
    public Circulo(double raio) {  
        super("Circulo");  
        this.raio = raio;  
    }  
  
    public double getRaio() { return raio; }  
  
    @Override  
    public double calcularArea() {  
        return Math.PI * raio * raio;  
    }  
}
```

Gabarito: Classe Retangulo

```
public class Retangulo extends Forma {
    private double largura;
    private double altura;

    public Retangulo(double largura, double altura) {
        super("Retangulo");
        this.largura = largura;
        this.altura = altura;
    }

    public double getLargura() { return largura; }
    public double getAltura() { return altura; }

    @Override
    public double calcularArea() {
        return largura * altura;
    }
}
```

Gabarito: Main com polimorfismo de formas

```
public class Main {  
    public static void main(String[] args) {  
        Forma[] formas = new Forma[2];  
  
        formas[0] = new Circulo(5.0);  
        formas[1] = new Retangulo(4.0, 6.0);  
  
        double total = 0;  
  
        for (int i = 0; i < formas.length; i++) {  
            System.out.println(formas[i]);  
            total += formas[i].calcularArea();  
        }  
  
        System.out.println("Area total: " + total);  
    }  
}
```

Saída esperada (formas)

```
Circulo | Area: 78.53981633974483  
Retangulo | Area: 24.0  
Area total: 102.53981633974483
```

O laço não sabe (nem precisa saber) se está lidando com um `Circulo` ou um `Retangulo` – ele só chama `calcularArea()` e confia no polimorfismo.

Problema:

- Crie uma classe abstrata `Veiculo` com:
 - atributos privados `modelo` e `velocidadeMaxima`;
 - construtor e *getters*;
 - método abstrato `tipoDeMotor()`;
 - `toString()` concreto, usando `tipoDeMotor()`.
- Crie as subclasses `Carro` (com portas) e `Moto` (com `temBau`).
- No `Main`, crie um vetor de `Veiculo` e imprima todos, usando polimorfismo.

Gabarito: Classe abstrata Veiculo

```
public abstract class Veiculo {  
    private String modelo;  
    private double velocidadeMaxima;  
  
    public Veiculo(String modelo, double velocidadeMaxima) {  
        this.modelo = modelo;  
        this.velocidadeMaxima = velocidadeMaxima;  
    }  
  
    public String getModelo() { return modelo; }  
    public double getVelocidadeMaxima() { return velocidadeMaxima; }  
  
    public abstract String tipoDeMotor();  
  
    @Override  
    public String toString() {  
        return modelo + " | Vel. Max: " + velocidadeMaxima +  
            " km/h | Motor: " + tipoDeMotor();  
    }  
}
```

Gabarito: Classe Carro

```
public class Carro extends Veiculo {
    private int portas;

    public Carro(String modelo, double velocidadeMaxima, int portas) {
        super(modelo, velocidadeMaxima);
        this.portas = portas;
    }

    public int getPortas() { return portas; }

    @Override
    public String tipoDeMotor() {
        return "Motor_a_combustao_(4_tempos)";
    }
}
```

Gabarito: Classe Moto

```
public class Moto extends Veiculo {
    private boolean temBau;

    public Moto(String modelo, double velocidadeMaxima, boolean
        temBau) {
        super(modelo, velocidadeMaxima);
        this.temBau = temBau;
    }

    public boolean isTemBau() { return temBau; }

    @Override
    public String tipoDeMotor() {
        return "Motor de duas rodas";
    }
}
```

Gabarito: Main com polimorfismo de veículos

```
public class Main {  
    public static void main(String[] args) {  
        Veiculo[] veiculos = new Veiculo[2];  
  
        veiculos[0] = new Carro("Civic", 210.0, 4);  
        veiculos[1] = new Moto("CB500", 180.0, false);  
  
        for (int i = 0; i < veiculos.length; i++) {  
            System.out.println(veiculos[i]);  
        }  
    }  
}
```

Saída esperada (veículos)

```
Civic | Vel. Max: 210.0 km/h | Motor: Motor a combustao (4 tempos)
CB500 | Vel. Max: 180.0 km/h | Motor: Motor de duas rodas
```

De novo: o Main trata Carro e Moto apenas como Veiculo – e cada um responde com seu próprio tipoDeMotor().

Parte 6 – Desafio Final

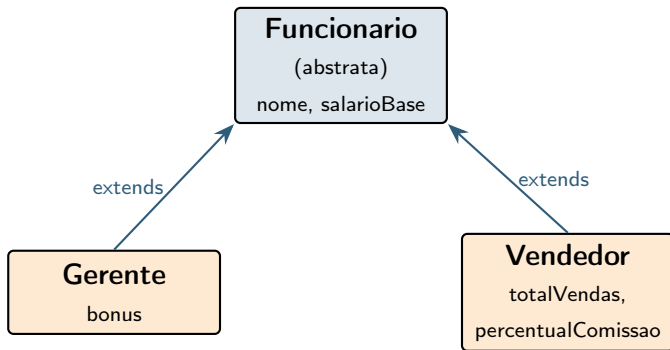
Agora é sua vez de generalizar uma hierarquia

Desafio final: Sistema de Folha de Pagamento

Sua missão:

- 1 Crie uma classe abstrata `Funcionario`, com atributos **privados** `nome` e `salarioBase`.
- 2 Adicione um construtor, *getters* e um método abstrato `calcularSalario()`.
- 3 Crie a subclasse `Gerente`, que recebe um `bonus` fixo somado ao salário base.
- 4 Crie a subclasse `Vendedor`, cujo salário depende do total de vendas e de um percentual de comissão.
- 5 Sobrescreva `calcularSalario()` de forma diferente em cada subclasse.
- 6 Crie também `toString()` em `Funcionario`, reaproveitando `calcularSalario()`.
- 7 No `Main`, crie um vetor de `Funcionario` com `Gerentes` e `Vendedores`, e imprima a folha de pagamento completa, usando **polimorfismo**.

Desafio Visual: Folha de Pagamento



Cada subclasse implementa `calcularSalario()` à sua maneira – e o Main consegue somar a folha inteira sem nunca verificar "se é Gerente ou Vendedor".